ReAct++: Enhancing Reasoning-Action Synergy through Few-Shot Prompting and Self-Consistency

Presenter:

- **Tiku Mohammd**

Supervisor:

- **Prof. Sourangshu Bhattacharya**

Department of Computer Science and Engineering,

Indian Institute of Technology Kharagpur

November 2025

Motivation

- Large Language Models (LLMs) such as **GPT** and **LLaMA** have shown impressive reasoning and text-generation capabilities.
- Yet, these models usually operate in a **static mode** - they only generate text, without interacting with or grounding their reasoning in the real world.
- In contrast, **real-world agents** must *reason*, *act*, and *adapt* to feedback from their environment.

The ReAct Framework (Reason + Act):

- Introduced the idea of **interleaving reasoning ("Thought") with actions ("Act")**.
- Enabled interpretability and step-by-step decision making.

Limitations of Original ReAct:

- Uses **single-shot prompting** → poor contextual understanding.
- **Inconsistent reasoning paths** → unstable behavior.
- Struggles with **complex, text-rich environments** like WebShop.

• Motivation for ReAct++:

This motivates the need for a more reliable framework that enhances reasoning - action synergy, improves contextual grounding, and ensures consistent decision-making across diverse tasks.

Problem Statement & Objectives

Problem Statement:

Existing reasoning-acting agents (like ReAct) show:

- **Inconsistent reasoning paths** across runs.
- **Limited contextual grounding** due to single-shot prompting.
- **Unstable decision-making** in complex web environments.

These issues reduce reliability and interpretability in real-world interactive tasks.

Research Objectives

1. **Enhance reasoning-action synergy**
   → Strengthen the logical link between model reasoning and corresponding environment actions.
2. **Improve contextual grounding**
   → Introduce *few-shot prompting* with multiple structured examples to guide more stable reasoning.
3. **Increase consistency and robustness**
   → Employ *self-consistency* by generating multiple reasoning paths and aggregating them through majority voting and semantic similarity.

Goal

→ To build a **robust, interpretable, and scalable reasoning-acting framework** that improves both reasoning stability and task success in web-interactive environments.

Research Contributions

Key Contributions of ReAct++: Our research makes the following key contributions toward improving reasoning-acting synergy in language-based agents :

1. **Enhanced Framework Design:**
   Developed *ReAct++*, integrating **Few-Shot Prompting** and **Self-Consistency** for stable reasoning and reliable action selection.

2. **Few-Shot Prompting Integration:**
   Introduced **multi-example contextual prompts** to strengthen generalization and reduce spurious actions.

3. **Self-Consistency Mechanism:**
   Implemented **multi-response reasoning** with **majority voting** and **semantic similarity aggregation** to minimize randomness.

4. **Empirical Evaluation:**
   Conducted extensive experiments on the **WebShop benchmark**, achieving
   → **+9.4% improvement** in Success Rate
   → **+21.6% increase** in average Score over baseline.

5. Automated Ground-Truth Pipeline:
Designed a **fully automated evaluation system** for 500 WebShop test instructions to ensure **reproducibility** and **scalability**.

ReAct++ demonstrates that:

structured prompting + ensemble-style reasoning = stronger reasoning-action synergy.

Background: The ReAct Framework

What is ReAct?

- *ReAct* = **Reason + Act** (Yao et al., 2023)
- A framework where language models **interleave reasoning ("Thought") & actions ("Act")** while interacting with an environment.
- Enables **interpretable decision-making** and **goal-directed behavior**.

The model alternates between:

- **Think:** Generate internal reasoning about the next step.
- **Act:** Perform an environment action (e.g., search, click).
- **Observe:** Read the result and continue reasoning.
- This design improves interpretability and allows dynamic interaction with external environments.

Advantages:

✅ Improves interpretability and reasoning clarity.
✅ Allows step-wise, grounded actions.

Limitations:

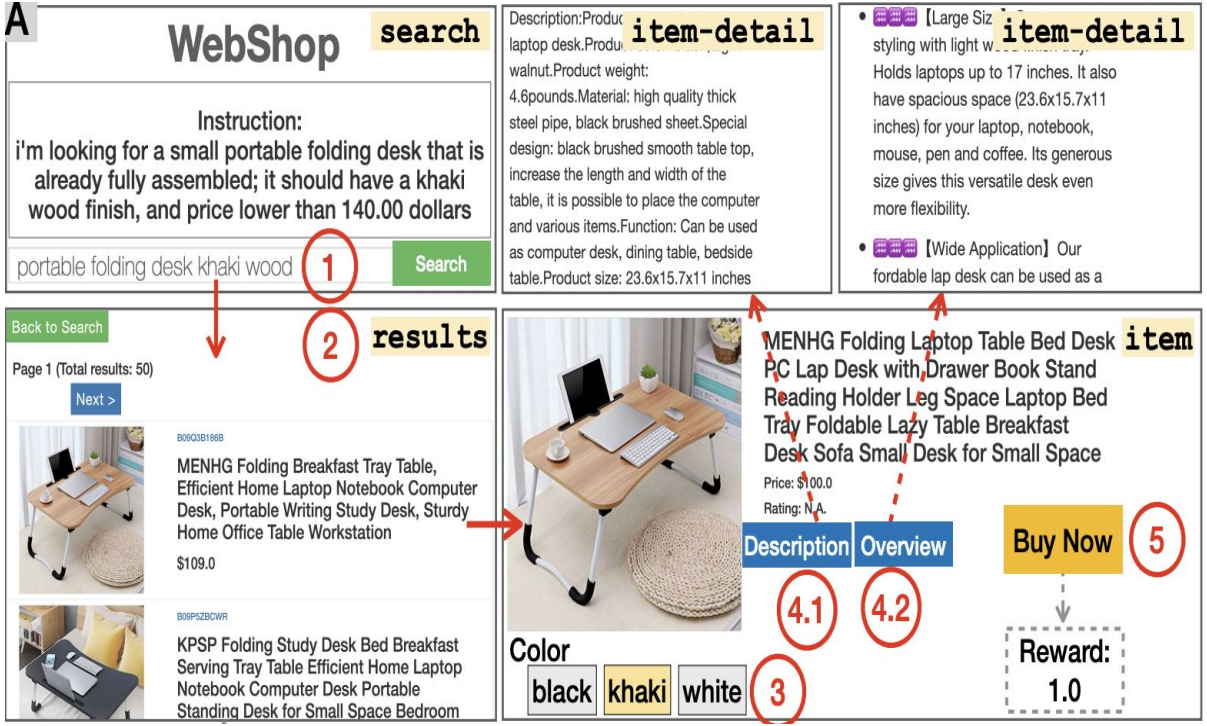
⚠️ Relies on **single-shot prompts** → poor contextual grounding.
⚠️ **Inconsistent reasoning** between runs.
⚠️ Struggles in long, multi-step environments (like WebShop).

The WebShop Environment

- A large-scale simulated **e-commerce platform** designed for grounded language interaction.

Contains:

- Over **1.1 million real-world products**
- **12,000 human-written instructions**
- Structured product attributes (price, color, size, etc.)
- Tasks require multi-step reasoning and action - searching, navigating, and selecting the correct product.
- Rewards are assigned based on how well the selected product matches the given instruction attributes.



ReAct++ Framework Overview

• **What is ReAct++?**

ReAct++ is an **enhanced reasoning-acting framework** that improves the original ReAct by integrating:

- **Few-Shot Prompting** → better contextual understanding.
- **Self-Consistency** → more stable and reliable reasoning.

• **Goal:**

- Strengthen the **synergy between reasoning and action** in LLM agents operating in grounded web environments.

• **Core Idea:**

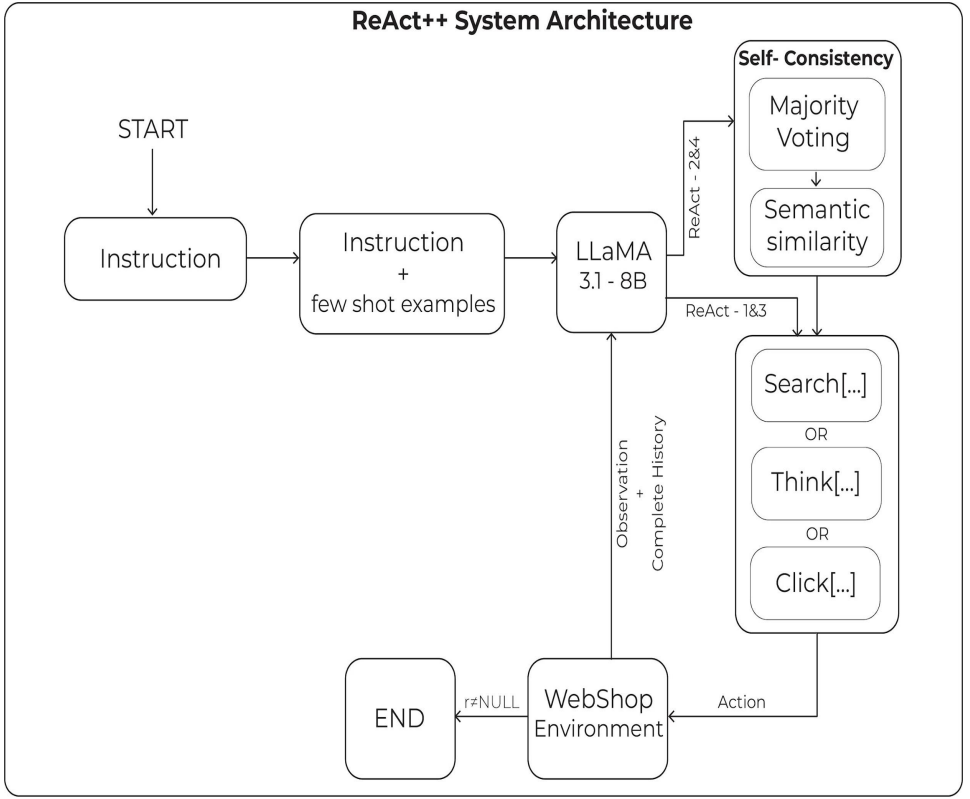
- Instead of one example (as in ReAct), ReAct++ uses **multiple in-context demonstrations** (few-shot).
- Generates **multiple reasoning trajectories**, then combines them using **voting and semantic similarity**.
- Produces **consistent and interpretable** decisions.

ReAct++ = *ReAct* + *Few-Shot Learning* + *Self-Consistent Reasoning*

System Architecture

ReAct++ Pipeline:

- **Input Instruction**
  → e.g., “Find a 3-oz bright citrus deodorant under $50.”
- **Few-Shot Prompting**
  → Two example reasoning-action trajectories are added to the prompt.
  → Gives the model structured guidance on *how* to think and act.
- **Reasoning-Acting Loop (Iterative Process)**
  - **Observation:** Environment returns webpage or product info.
  - **Thought:** Model reasons internally (think[...]).
  - **Action:** Executes commands (search[...], click[...]).
  - Repeat until goal reached or max steps (15).
- **Self-Consistency Module (for SC variants)**
  - Generate *n* reasoning paths (default: 10).
  - Aggregate outputs using:
    - **Majority voting** → for discrete actions (click/buy).
    - **Semantic similarity clustering** → for reasoning & search queries.
- **Final Action Execution**
  → The stable, aggregated action is sent to the environment.



Methodology Overview

- **1. Problem Formulation**
- The WebShop task is modeled as a **Partially Observable Markov Decision Process (POMDP)**.
- Components:
- **State (s):** Current environment observation (product list, page view).
- **Action (a):** Model's decision (search, click, buy, think).
- **Observation (o):** Feedback received after taking an action.
- **Reward (r):** Final score measuring how well the chosen product satisfies the instruction.
- **Policy (π):** The language model generating actions conditioned on reasoning and observations.
- **Objective:**
  Maximize expected cumulative reward

$$\mathbb{E}_{\pi}\left[\sum_{t} R\left(s_t, a_t\right)\right]$$

  by optimizing reasoning–acting sequences.

2. ReAct++ Methodology

The methodology involves four main stages integrated into an iterative reasoning-acting loop:

1. **Reasoning Generation (Think Step):**
   • The model generates internal reasoning traces to plan the next action.

2. **Action Execution:**
   • Executes actions like search[...] or click[...] within the WebShop environment.

3. **Observation Processing:**
   • Reads the environment's response (product listings, options, prices) and appends it to the prompt.

4. **Self-Consistency Aggregation:**
   • Generates multiple reasoning-action responses.
   • Applies **majority voting** for discrete actions and **semantic similarity clustering** for reasoning steps.
   • Selects the most representative action to proceed with.

3. Few-Shot Prompting Setup

- Each prompt includes **two demonstration examples** illustrating complete reasoning-action sequences.
- Helps the model learn structured decision patterns and generalize to unseen instructions.
- Reduces irrelevant or inconsistent reasoning during inference.

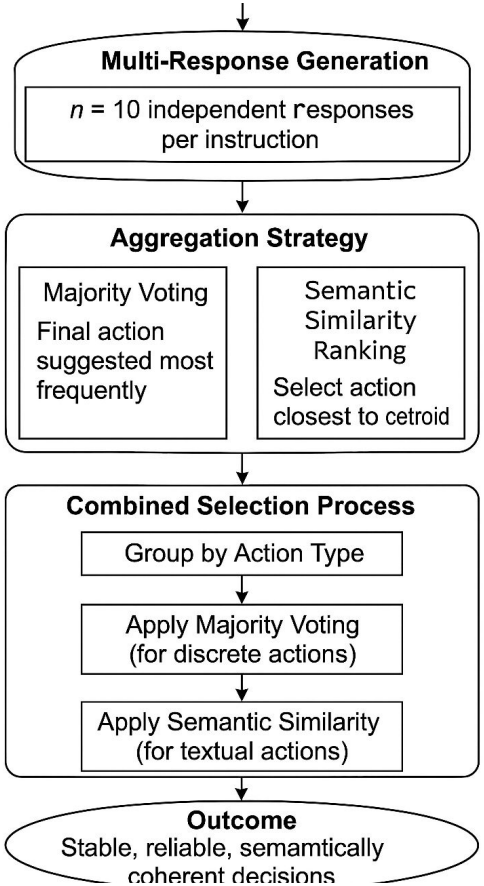
4. Self-Consistency Mechanism

1. Motivation

- In the original ReAct framework, a single reasoning path is generated for each instruction.
- This often leads to **inconsistent or suboptimal actions** due to randomness in LLM outputs.
- To address this, ReAct++ introduces a **Self-Consistency (SC)** mechanism inspired by ensemble reasoning.
- The goal is to make decisions **more stable**, **reliable**, and **semantically coherent**.

2. Multi-Response Generation

- For each user instruction, the model generates **n = 10 independent reasoning-action paths**.
- Each path may produce a slightly different proposed action or reasoning trace.
- Instead of selecting the first output, ReAct++ aggregates all responses to identify the most consistent one.



3. Aggregation Strategy
The aggregation combines two complementary techniques:

a. Majority Voting (for discrete actions)
- Used when the model's output is categorical (e.g., click[...], buy[...]).
- The final action is selected as the one **most frequently suggested** across all reasoning paths.
- Example:
  If 6 out of 10 responses suggest click[B078GWRC1J], that action is chosen.

b. Semantic Similarity Ranking (for reasoning and search actions)
- Used when responses are textual, such as reasoning steps or search[...] queries.
- Each reasoning phrase is embedded using **SentenceTransformer ('all-MiniLM-L6-v2')**.
- Cosine similarity is computed between all embeddings, and the action **closest to the centroid** is chosen as the final one.
- This ensures semantically aligned reasoning even when wordings differ.

5. Outcome
- The hybrid approach balances **quantitative consensus** (majority voting) with **qualitative semantic alignment**.
- Reduces random variability and enhances interpretability of reasoning traces.
- Leads to higher **success rate** and **decision stability** across multiple trials.

Model Variants

To evaluate the effect of structured prompting and self-consistency, four model variants were designed under the ReAct++ framework.
Each variant incrementally builds upon the previous one to isolate the impact of each enhancement.

1. ReAct-1 (Baseline)

- Single-example (1-shot) prompt.
- Traditional ReAct structure using a single reasoning-action demonstration.
- No multi-response or consistency aggregation.
- Serves as the **baseline implementation** for comparison.

2. ReAct-2 (Few-Shot Prompting)

- Uses **two demonstration examples (2-shot prompt)** instead of one.
- Provides the model with additional structured reasoning-action trajectories.
- Improves contextual understanding and reasoning stability.
- Encourages better generalization across unseen tasks.

3. ReAct-1SC (Self-Consistency with 1-Shot Prompt)

- Uses the **1-shot prompt** setup but enables **multi-response generation**.
- Generates multiple reasoning paths (n = 10 completions).
- Aggregates outcomes using:
- **Majority Voting** for discrete actions.
- **Semantic Similarity Ranking** for reasoning and search steps.
- Improves decision reliability through ensemble-style reasoning.

4. ReAct-2SC (Few-Shot + Self-Consistency)

- Combines **two-shot prompting** with **multi-response aggregation**.
- Generates multiple responses per instruction (n = 10) and selects the best action through hybrid aggregation.
- Achieves the **best performance** among all variants - higher success rate and more stable reasoning traces.

Variant	Prompt Type	No. of Examples	Self-Consistency	Description
ReAct-1	1-shot	1	✗	Baseline ReAct implementation
ReAct-2	Few-shot	2	✗	Structured examples improve grounding
ReAct-1SC	1-shot	1	✓	Multi-response reasoning with voting
ReAct-2SC	Few-shot	2	✓	Best variant: Stable, grounded, consistent

The transition from **ReAct-1 → ReAct-2SC** shows the evolution from a simple single-example reasoning agent to a **multi-example, ensemble-consistent agent** capable of more reliable decision-making in the WebShop environment.

Experimental Setup

1. Model Details

- **Model Used:** Meta-LLaMA-3.1-8B-Instruct
- **Type:** Instruction-tuned open-weight large language model
- **Quantization:** 4-bit precision using BitsAndBytes configuration for memory-efficient inference
- **Inference Framework:** Hugging Face Transformers
- **Computation Mode:** Inference-only (no fine-tuning)

2. Hardware Configuration

- **GPUs:** 4 × NVIDIA A16 (15 GB VRAM each)
- **Driver Version:** 580.65.06
- **CUDA Version:** 13.0
- **Runtime Environment:** Python 3.10, PyTorch 2.2, Transformers 4.41+
- **Execution Mode:** Distributed across 4 GPUs using device_map="auto"

3. Dataset : WebShop Benchmark

- **Domain:** Simulated e-commerce platform for grounded web interaction.
- **Dataset Size:**
- 1.1 million real product listings
- 12,000 natural-language instructions
- **Test Split:** 500 instructions automatically sampled and evaluated.
- **Task:** Find and purchase the correct item following a user's textual instruction.

4. Evaluation Metrics

- **Reward (r):**
  Based on alignment between selected product and target attributes/options.

$$r = r_{type} \cdot \frac{|\ U_{att} \cap Y_{att}\ | + |\ U_{opt} \cap Y_{opt}\ | + 1[y_{price} \leq u_{price}]}{|\ U_{att}\ | + |\ U_{opt}\ | + 1}$$

- **Task Score:**

$$\text{Score} = 100 \times \textit{average reward}$$

- **Success Rate (SR):**
  Fraction of test cases with reward = 1 (perfect goal satisfaction).

Parameter	Value
Max tokens per step	100
Temperature	0.9 (for SC) / 0.0 (baseline)
Top-p	0.9
Top-k	50
Num responses	1 or 10
Context length	6400 characters
Stop token	Newline (\n)

5. Tools & Libraries

- **Transformers (Hugging Face):** Model loading and inference
- **BitsAndBytes:** 4-bit quantization
- **SentenceTransformers:** Embedding-based semantic aggregation
- **NumPy, Torch:** Supporting numerical computations
- **Custom WebShop Environment:** For interaction and evaluation

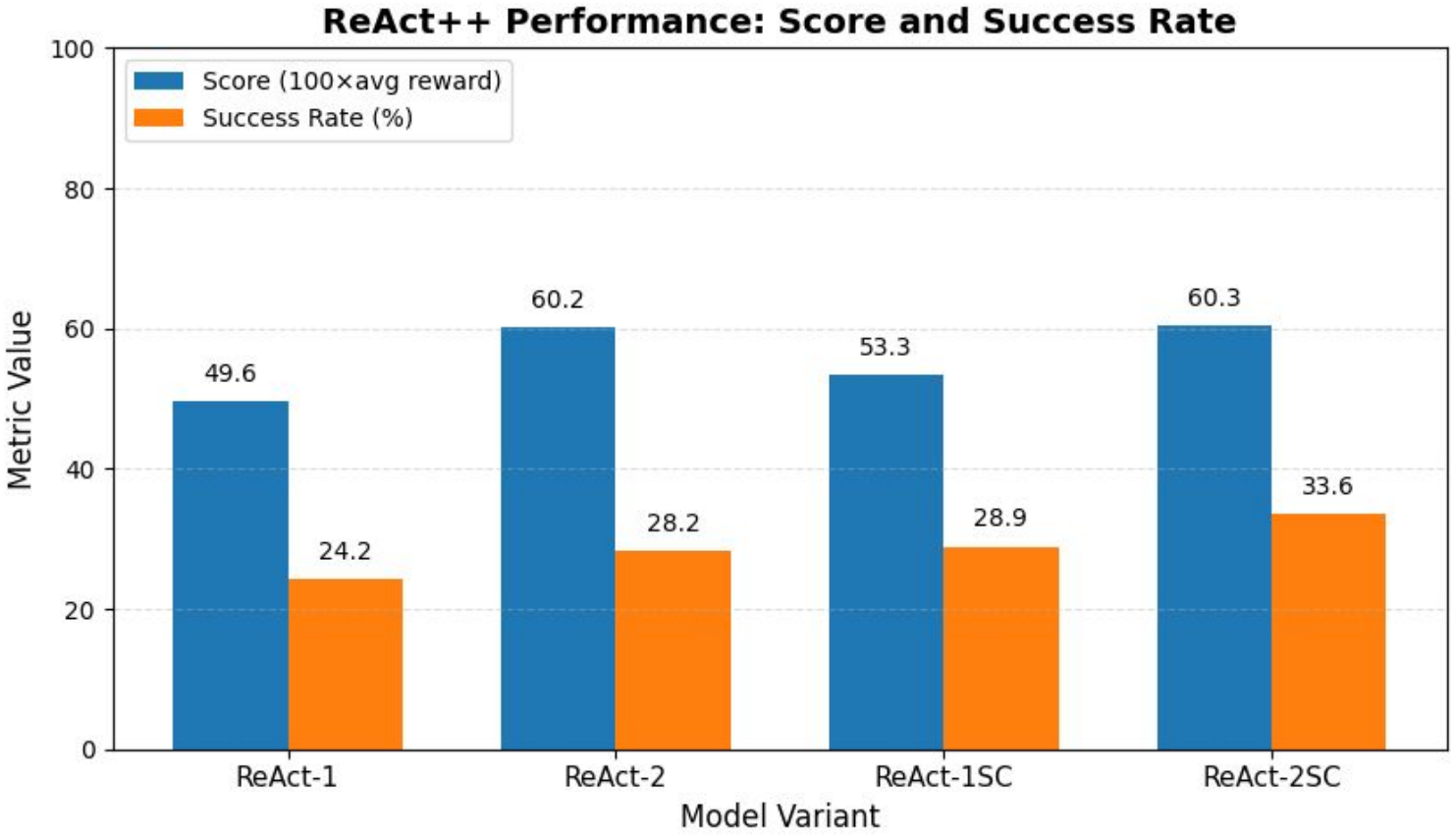
Results and Performance Comparison

Overview

- Four ReAct++ variants were evaluated on **500 WebShop test instructions**.
- The models were compared using two key metrics:
- **Task Score (100 × Average Reward)**
- **Success Rate (SR)** : fraction of episodes with reward = 1.

Quantitative Results:

Model Variant	Model	Score	Success Rate (SR)
Original ReAct (GPT)	GPT	63.79	35.8%
ReAct-1 (LLaMA, baseline)	LLaMA-3.1-8B	49.6	24.2%
ReAct-2 (Few-Shot)	LLaMA-3.1-8B	60.2	28.2%
ReAct-1SC (Self-Consistency, 1-shot)	LLaMA-3.1-8B	53.3	28.9%
ReAct-2SC (Few-Shot + Self-Consistency)	LLaMA-3.1-8B	60.3	33.6%


ReAct++ Performance: Score and Success Rate
Score (100×avg reward)
Success Rate (%)
Metric Value
Model Variant
ReAct-1
ReAct-2
ReAct-1SC
ReAct-2SC
49.6
24.2
60.2
28.2
53.3
28.9
60.3
33.6
0
20
40
60
80
100

1. Few-Shot Prompting Impact

- Moving from **1-shot (ReAct-1)** to **2-shot (ReAct-2)** increased both *Score* and *Success Rate*.
- Structured examples provided **better contextual grounding** and improved the model's understanding of reasoning-action sequences.
- The improvement of **+10.6 in Score** and **+4.0% in SR** highlights the benefit of additional in-context examples.

2. Self-Consistency Impact

- **ReAct-1SC** outperforms the baseline despite using the same 1-shot prompt.
- The improvement comes from **aggregating multiple reasoning paths**, reducing randomness in actions.
- Achieved **+4.7% SR** improvement due to more stable and interpretable decision-making.

3. Combined Effect (ReAct-2SC)

- **ReAct-2SC**, combining *Few-Shot Prompting* and *Self-Consistency*, achieved the **highest performance**:
- **Score:** 60.3
- **Success Rate:** 33.6%
- This demonstrates that structured prompting (for context) and ensemble reasoning (for consistency) are **complementary and synergistic**.

4. Key Takeaways

- ReAct++ improves **reasoning stability**, **contextual understanding**, and **task success**.
- The combination of few-shot prompting and self-consistency transforms ReAct from a static reasoning agent into a **robust interactive decision-maker**.
- Consistent performance gains validate the **scalability and reliability** of the proposed framework.

Ablation and Sensitivity Analysis

1. Purpose

❖ To evaluate how sensitive **ReAct++** performance is to changes in key parameters such as:

- Number of **Self-Consistency samples (n)**
- **Temperature** (sampling diversity)

❖ Helps understand **trade-offs between consistency, diversity, and accuracy**.

n (samples)	Score	Success Rate (SR)	Observation
1 (no SC)	49.6	24.2%	Baseline, unstable reasoning
5	56.8	30.4%	Clear improvement; ensemble reduces random errors
10	60.3	33.6%	Peak performance; optimal balance of diversity & stability
15	60.0	33.2%	Slight drop; redundancy adds computational cost

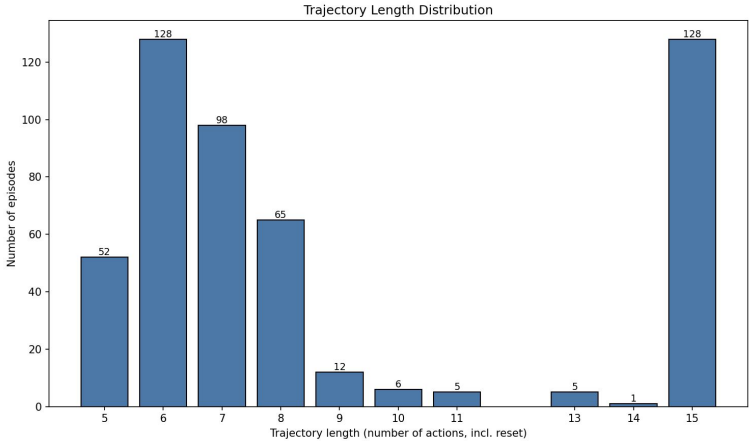
Insight: Increasing *n* improves consistency up to a saturation point (≈10 samples).

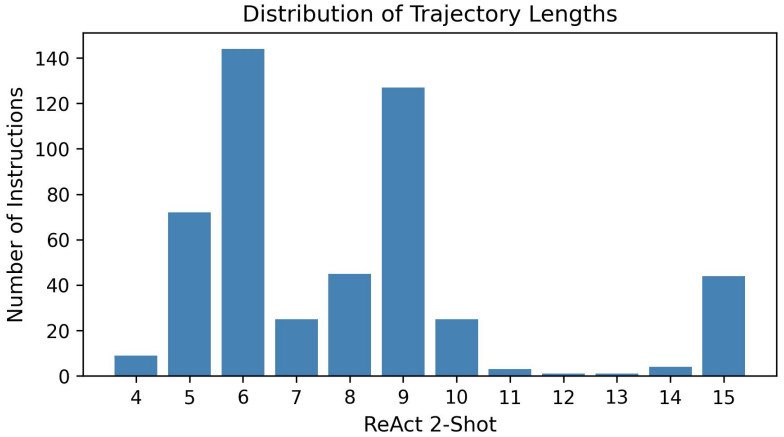
Trajectory and Score Distribution Analysis

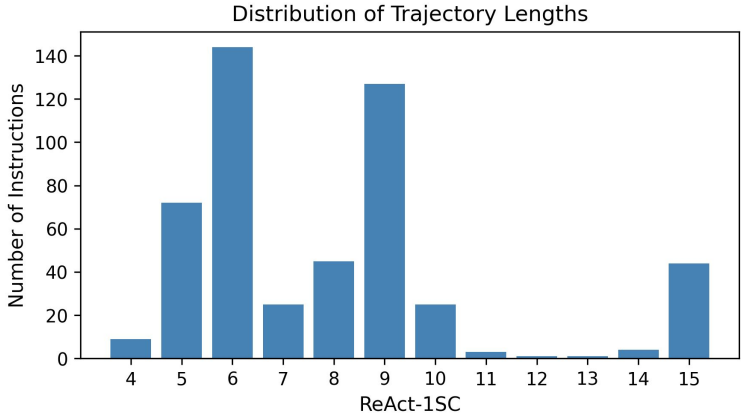
- To complement quantitative results, trajectory-level analysis was conducted across all ReAct++ variants.
- **Trajectory length** represents the total number of reasoning-action steps before task completion or termination.
- This analysis helps evaluate **efficiency** (fewer steps → faster convergence) and **stability** (consistent reasoning paths).
- Each instruction was categorized based on its **final score**:

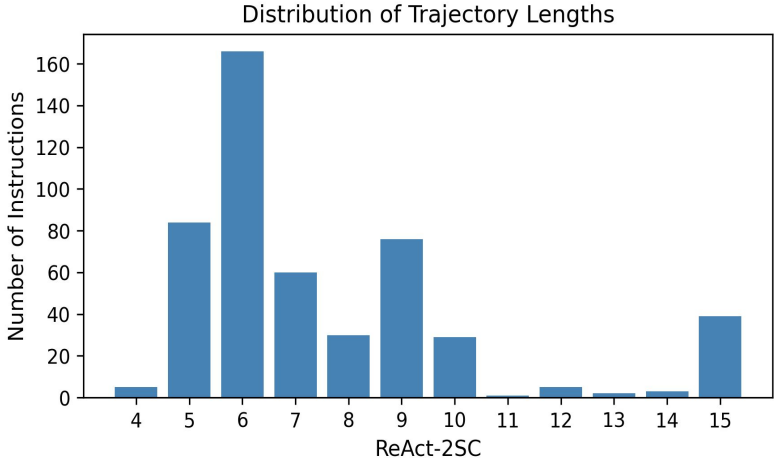
    - **Score = 1.0** → Perfect match (complete success)
    - **Score between 0 and 1** → Partial match (some conditions met)
    - **No Score (0)** → Incomplete or failed trajectory

- **Key Takeaway:**
  ReAct++ variants show progressively **shorter, more stable trajectories**, and **fewer failures**, indicating improved reasoning-action efficiency.


Trajectory Length Distribution
Number of episodes
Trajectory length (number of actions, incl. reset)
52
128
98
65
12
6
5
5
1
128


Distribution of Trajectory Lengths
Number of Instructions
ReAct 2-Shot


Distribution of Trajectory Lengths
Number of Instructions
ReAct-1SC


Distribution of Trajectory Lengths
Number of Instructions
ReAct-2SC

• Trajectory length measures the number of reasoning-action steps taken before the task terminates (success, partial match, or failure).
- The maximum allowed trajectory length in WebShop is **15 steps**.
- The distribution analysis highlights how efficiently each model reaches the correct decision.

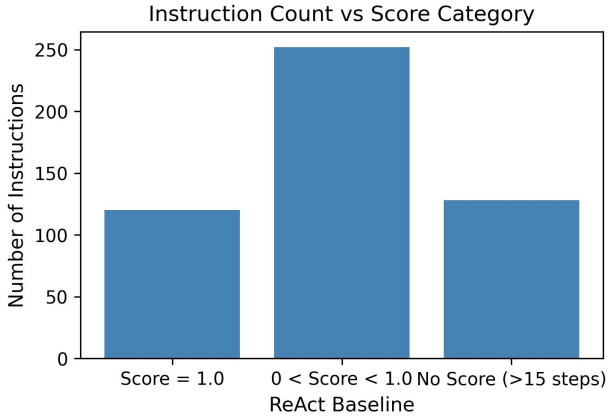
Original ReAct (Baseline):
- Most successful instructions complete within **5-8 steps**.
- However, **128 instructions** reached the full **15-step limit**, indicating failed or looping trajectories.

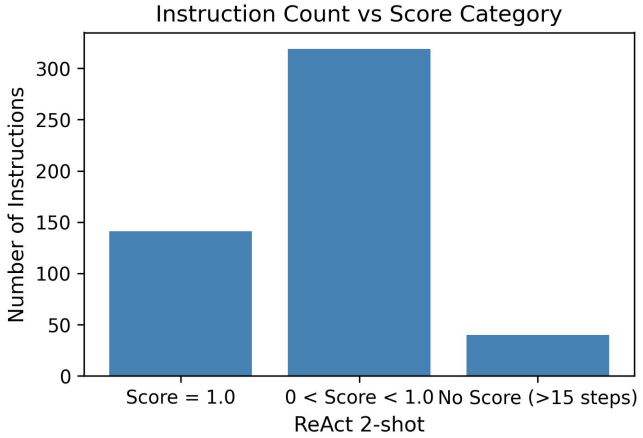
ReAct (2-shot):
- Majority of trajectories converge between **5-9 steps**.
- Only **40 instructions** hit the 15-step limit - a significant reduction from the baseline.
- Demonstrates improved contextual understanding and fewer redundant actions.

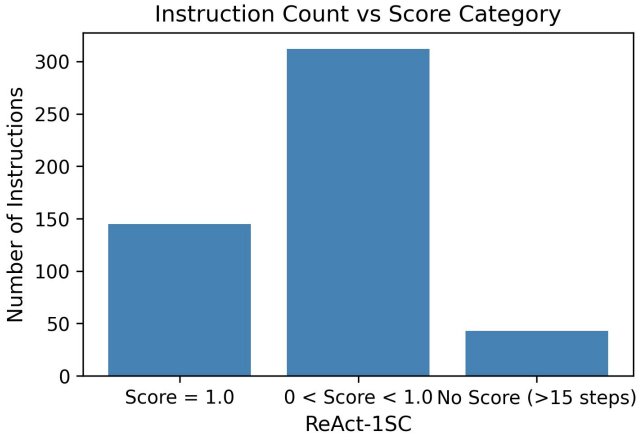
ReAct-1SC (Self-Consistency 1-shot):
- Similar compact distribution with most trajectories in the **5-9 step range**.
- **43 instructions** reached 15 steps, showing that self-consistency improves decision reliability even without extra examples.

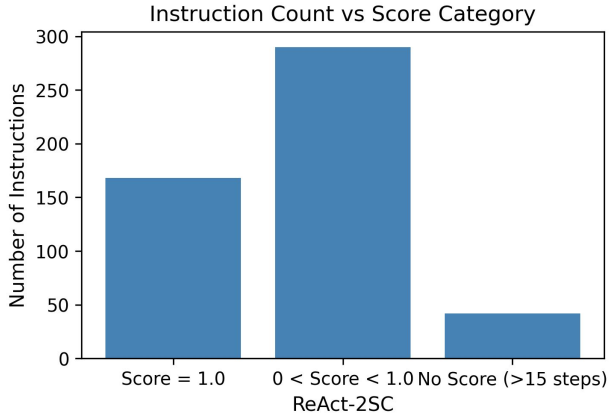
ReAct-2SC (Self-Consistency 2-shot):
- Most efficient and stable behavior among all variants.
- Majority of successful runs terminate within **5-9 steps**, and only **42 cases** reached the 15-step limit.
- Indicates optimal reasoning–action alignment and strong convergence behavior.

Key Takeaways:
- Few-shot prompting and self-consistency both lead to **shorter, more efficient trajectories**.
- The number of failed (max-step) episodes decreases consistently across variants.
- **ReAct-2SC** achieves the best trade-off - faster convergence and minimal failures.


Instruction Count vs Score Category
Number of Instructions
0
50
100
150
200
250
Score = 1.0
0 < Score < 1.0
No Score (>15 steps)
ReAct Baseline


Instruction Count vs Score Category
Number of Instructions
0
50
100
150
200
250
300
Score = 1.0
0 < Score < 1.0
No Score (>15 steps)
ReAct 2-shot


Instruction Count vs Score Category
Number of Instructions
0
50
100
150
200
250
300
Score = 1.0
0 < Score < 1.0
No Score (>15 steps)
ReAct-1SC


Instruction Count vs Score Category
Number of Instructions
0
50
100
150
200
250
300
Score = 1.0
0 < Score < 1.0
No Score (>15 steps)
ReAct-2SC

Score Distribution Across Variants (500 Instructions)

Variant	Score = 1.0	0 < Score < 1	No Score (>15 steps)
ReAct-1 (Baseline)	120	252	128
ReAct-2 (Few-shot)	141	319	40
ReAct-1SC (Self-consistent 1-shot)	145	312	43
ReAct-2SC (Self-consistent 2-shot)	**168**	**290**	**42**

3. Temperature Sensitivity

Temperature	Score	Success Rate (SR)	Observation
0.0	49.6	24.2%	Deterministic, low diversity
0.7	57.5	30.9%	Balanced diversity and reasoning stability
0.9	60.3	33.6%	Optimal for self-consistent aggregation
1.1	58.1	31.0%	Too diverse, causes noise in reasoning

Insight: Moderate sampling temperature (~0.9) yields the best results - enough diversity to support self-consistency without destabilizing reasoning.

4. Key Findings

- Ensemble-style **self-consistency** is the most impactful factor for improving success rate.
- **Few-shot prompting** provides grounding, while **aggregation parameters** control reliability.
- ReAct++ achieves optimal performance at:
  → *n = 10*, *temperature = 0.9*.
- These settings deliver **maximum stability with minimal computational overhead**.

Why it Works

1. Why Few-Shot Helps

- **Provides structural guidance:**
  Few-shot prompts give the model concrete examples of how to alternate between reasoning and acting.
- **Improves contextual grounding:**
  Multiple demonstrations expose the model to different reasoning-action patterns, allowing better understanding of instruction-response relationships.
- **Enhances generalization:**
  The model learns reusable reasoning templates that improve performance on unseen queries.
- **Reduces error propagation:**
  With more examples, the model maintains coherent reasoning across multi-step tasks instead of drifting from context.
- **Observation:**
  ReAct-2 variants consistently outperform ReAct-1, confirming that richer contextual cues enhance reasoning quality and stability.

2. Why Self-Consistency Helps

- **Reduces stochastic variance:**
  By sampling multiple reasoning paths (n=10), random noise in model outputs is averaged out.
- **Aggregates collective reasoning:**
  Ensemble-style voting ensures that only the most consistent and semantically aligned reasoning paths are selected.
- **Improves decision reliability:**
  Prevents premature or inconsistent actions by emphasizing consensus among multiple reasoning traces.
- **Stabilizes outcomes:**
  Encourages interpretable and reproducible behavior - crucial for real-world deployment.

Observation:
ReAct-1SC and ReAct-2SC achieve higher and more stable success rates compared to non-SC counterparts.

Failure Cases

Despite overall improvement, ReAct++ still faces limitations in complex scenarios:

Failure Type	Description	Cause / Limitation
Option mismatch	Incorrect attribute or variant selected	Ambiguity in product options (e.g., color, size)
Noisy product descriptions	LLM misinterprets textual noise	Lack of structured metadata
Search engine constraints	Relevant items not retrieved	Incomplete or imperfect query formulation
Context truncation	Long interactions lose prior information	Prompt limited to 6400 characters

Conclusion

This work proposed **ReAct++**, an enhanced reasoning-acting framework for large language models.

The framework strengthens the original ReAct paradigm through two core improvements:

- **Few-Shot Prompting** - introduces structured, multi-example reasoning-action patterns for contextual grounding.
- **Self-Consistency Mechanism** - aggregates multiple reasoning paths using majority voting and semantic similarity for decision stability.

Experiments with **LLaMA-3.1-8B-Instruct** showed significant performance improvements:

- **+9.4%** increase in *Success Rate*
- **+10.7** improvement in *Task Score* over the baseline ReAct model.

Results demonstrate that **ReAct++** achieves more grounded, consistent, and interpretable reasoning-acting behavior in dynamic web environments.

Key Takeaways

- ReAct++ effectively bridges the gap between **reasoning quality** and **action reliability**.
- Structured few-shot prompts provide **contextual scaffolding**, while ensemble-style reasoning enhances **robustness**.
- The proposed framework can serve as a foundation for **scalable, web-interactive LLM agents**.

Future Work

- **Extended Few-Shot Scaling:**
  Explore 3-shot, 4-shot, and variable-shot prompting to further enhance contextual diversity.
- **Cross-Environment Generalization:**
  Evaluate ReAct++ on other interactive settings such as **ALFWorld** or **MiniWoB++**.
- **Context Compression & Memory:**
  Integrate retrieval-augmented memory to handle long-horizon reasoning beyond 6400 tokens.
- **Improved Option Alignment:**
  Use semantic search or structured product metadata to reduce attribute mismatches.
- **Model Optimization:**
  Experiment with larger models (e.g., LLaMA-70B) and efficient inference techniques for scaling.

“ReAct++ demonstrates that structured reasoning and self-consistent decision-making can transform large language models from static text generators into reliable, goal-oriented intelligent agents.”

Automated Ground Truth Generation

1. Objective
To automatically generate reliable **ground-truth labels** for 500 WebShop test instructions without manual intervention.

This was achieved using a **leveled search and paraphrasing algorithm** that explores multiple search variations to find the best product match.

2. Algorithm Overview
Step 1 - Initialization

- For each **session ID** (fixed_0, fixed_1, …):
- Reset the WebShop environment.
- Extract the **original user instruction**.
- Use **LLaMA** to perform simulated actions (limited to 3 products).
- Log all actions and observations (text + JSON).
- Output: reward, paraphrased instruction, and original instruction.

Step 2 - Ground Truth Phase (Leveled Search)

Level 1: Original Instruction

- Search using the **original user instruction**.
- Explore first **3 pages of results**.
- Test all product-option combinations.
- If any product achieves **reward = 1.0** → **STOP** (Ground Truth = original instruction).

Level 2: Paraphrased Keywords

- If Level 1 fails:
- Call get_paraphrases(level=2, history=[original_instruction]).
- **LLaMA** generates 50 alternative search keywords.
- **GloVe embeddings** convert these to dense vectors.
- Apply **K-Means clustering** to group into 5 clusters.
- Extract **1 representative keyword** from each cluster.
- Search using each of the 5 representative keywords across 3 pages.
- If reward = 1.0 → STOP (Ground Truth = that keyword).

Level 3: Extended Paraphrasing

- If Level 2 fails:
- Call get_paraphrases(level=3, history=[all previous keywords]).
- Generate another set of 50 keywords, cluster into 5 new representatives.
- Repeat the search and combination testing as in Level 2.
- If reward = 1.0 → STOP (Ground Truth = that keyword).

Step 3 - Logging and Output

- Log all search trajectories, paraphrased keywords, and successful products to **JSON** and **TXT** files.
- Store:
- session_id
- original_instruction
- ground_truth_keyword
- reward
- winning_trajectory
- searched_keywords_by_level
- Output separated into **ground_truth_success.json** and **ground_truth_failed.json**

4. Summary

- Multi-level search with **semantic paraphrasing** ensures that if a correct match exists, it will be found within 3 levels.
- Fully automated and reproducible - replaces manual labeling.
- Ensures consistent ground-truth quality for large-scale experiments.

References

Core Research Papers

1. **Yao et al., 2023** - *ReAct: Synergizing Reasoning and Acting in Language Models*
   arXiv preprint arXiv:2210.03629
2. **Yao et al., 2022** - *WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents*
   arXiv preprint arXiv:2207.01206
3. **Wang et al., 2022** - *Self-Consistency Improves Chain-of-Thought Reasoning in Language Models*
   arXiv preprint arXiv:2203.11171

Tools and Frameworks

4. **Meta AI (2024)** - *LLaMA 3.1 Model Card*
   Available at: https://ai.meta.com/llama
5. **Dettmers et al., 2022** - *BitsAndBytes: 8-bit and 4-bit Quantization for Transformers*
   GitHub: https://github.com/TimDettmers/bitsandbytes
6. **Reimers & Gurevych, 2019** - *Sentence-Transformers: Multilingual Sentence Embeddings using BERT*
   https://www.sbert.net
7. **Wolf et al., 2020** - *Transformers: State-of-the-Art Natural Language Processing*
   GitHub: https://github.com/huggingface/transformers

Thank You!

Questions and Discussion